Table 16: The implementation algorithm of the proposed PPU_5 in the signed array multiplier.

Step	Operation	Equivalent Logic
1	$S_1 = 0$	$FALSE(S_1)$
2	$S_2 = 0$	$FALSE(S_2)$
3	$S_3 = 0$	$FALSE(S_3)$
4	$S_4 = 0$	$FALSE(S_4)$
5	$(b \to 0) \equiv (b \to S_1) = S_1{}'$	$S_1{}' = \bar{b}$
6	$(a \to S_1{}') = S_1{}''$	$S_1{}'' = \bar{a} + \bar{b} = \alpha$
7	$(d \to 0) \equiv (d \to S_2) = S_2{}'$	$S_2{}' = \bar{d}$
8	$(c \to S_2{}') = S_2{}''$	$S_2{}'' = \bar{c} + \bar{d} = \beta$
9	$(S_1{}'' \to 0) \equiv (S_1{}'' \to S_3) = S_3{}'$	$S_3{}' = ab = \bar{\alpha}$
10	$(S_2{}'' \to 0) \equiv (S_2{}'' \to S_4) = S_4{}'$	$S_4{}' = cd = \bar{\beta}$
11	$(S_3{}' \to S_2{}'') = S_2{}'''$	$S_2{}''' = (\bar{a} + \bar{b}) + (\bar{c} + \bar{d})$
12	$(S_1{}'' \to S_4{}') = S_4{}''$	$S_4{}'' = ab + cd$
13	$S_1 = 0$	$FALSE(S_1)$
14	$(S_2{}''' \to 0) \equiv (S_2{}''' \to S_1) = S_1{}'$	$S_1{}' = ab \cdot cd$
15	$(S_4{}'' \to S_1{}') = S_1{}''$	$S_1{}'' = (\bar{a} + \bar{b}) \cdot (\bar{c} + \bar{d}) + ab \cdot cd = \overline{\alpha \oplus \beta}$
16	$S_3 = 0$	$FALSE(S_3)$
17	$(C_{in} \to 0) \equiv (C_{in} \to S_3) = S_3{}'$	$S_3{}' = \overline{C_{in}}$
18	$(S_4{}'' \to C_{in}) = C_{in}{}'$	$C_{in}{}' = (\bar{a} + \bar{b}) \cdot (\bar{c} + \bar{d}) + C_{in}$
19	$(S_1{}'' \to S_3{}') = S_3{}''$	$S_3{}'' = (\alpha \oplus \beta) + \overline{C_{in}}$
20	$S_4 = 0$	$FALSE(S_4)$
21	$(S_3{}'' \to 0) \equiv (S_3{}'' \to S_4) = S_4{}'$	$S_4{}' = \overline{\alpha \oplus \beta} \cdot C_{in}$
22	$S_1 = 0$	$FALSE(S_1)$
23	$(C_{in}{}' \to 0) \equiv (C_{in}{}' \to S_1) = S_1{}'$	$S_1{}' = (ab + cd) \cdot \overline{C_{in}}$
24	$(S_2{}''' \to S_1{}') = S_1{}''$	$S_1{}'' = ab \cdot cd + (ab + cd) \cdot \overline{C_{in}}$ $= \bar{\alpha} \cdot \bar{\beta} + (\bar{\alpha} + \bar{\beta}) \cdot \overline{C_{in}}$
25	$(S_2{}''' \to C_{in}{}') = C_{in}{}''$	$C_{in}{}'' = ab \cdot cd + \left((\bar{a} + \bar{b}) \cdot (\bar{c} + \bar{d}) + C_{in}\right)$ $= \bar{\alpha} \cdot \bar{\beta} + (\alpha \cdot \beta + C_{in})$
26	$(C_{in}{}'' \to S_4{}') = S_4{}''$	$S_4{}'' = \left[(\alpha + \beta) \cdot \left((\bar{\alpha} + \bar{\beta}) \cdot \overline{C_{in}}\right)\right] + (\overline{\alpha \oplus \beta} \cdot C_{in})$ $= (\alpha \oplus \beta \cdot \overline{C_{in}}) + (\overline{\alpha \oplus \beta} \cdot C_{in})$ $= \alpha \oplus \beta \oplus C_{in} = \boldsymbol{Sum}$
27	$S_3 = 0$	$FALSE(S_3)$
28	$(S_1{}'' \to 0) \equiv (S_1{}'' \to S_3) = S_3{}'$	$S_3{}' = (\alpha + \beta) \cdot (\alpha \cdot \beta + C_{in})$ $= \alpha \cdot \beta + C_{in}(\alpha + \beta) = \boldsymbol{C_{out}}$

Table 17: The implementation algorithm of the proposed PPU_6 in the signed array multiplier.

Step	Operation	Equivalent Logic
1	$S_1 = 0$	$FALSE(S_1)$
2	$S_2 = 0$	$FALSE(S_2)$
3	$S_3 = 0$	$FALSE(S_3)$
4	$(b \to 0) \equiv (b \to S_1) = S_1{}'$	$S_1{}' = \bar{b}$
5	$(a \to S_1{}') = S_1{}''$	$S_1{}'' = \bar{a} + \bar{b} = \alpha$
6	$(\beta \to 0) \equiv (\beta \to S_2) = S_2{}'$	$S_2{}' = \bar{\beta}$
7	$(S_1{}'' \to S_2{}') = S_2{}''$	$S_2{}'' = ab + \bar{\beta}$
8	$(S_1{}'' \to 0) \equiv (S_1{}'' \to S_3) = S_3{}'$	$S_3{}' = ab = \bar{\alpha}$
9	$S_1 = 0$	$FALSE(S_1)$
10	$(S_2{}'' \to 0) \equiv (S_2{}'' \to S_1) = S_1{}'$	$S_1{}' = (\bar{a} + \bar{b}) \cdot \beta$
11	$(S_3{}' \to \beta) = \beta'$	$\beta' = (\bar{a} + \bar{b}) + \beta$
12	$(\beta' \to S_1{}') = S_1{}''$	$S_1{}'' = ab \cdot \bar{\beta} + (\bar{a} + \bar{b}) \cdot \beta$
13	$S_3 = 0$	$FALSE(S_3)$
14	$(C_{in} \to 0) \equiv (C_{in} \to S_3) = S_3{}'$	$S_3{}' = \overline{C_{in}}$
15	$(S_1{}'' \to S_3{}') = S_3{}''$	$S_3{}'' = \left((\bar{a} + \bar{b}) + \beta\right) \cdot (ab + \bar{\beta}) + \overline{C_{in}}$
16	$S_1 = 0$	$FALSE(S_1)$
17	$(S_3{}'' \to 0) \equiv (S_3{}'' \to S_1) = S_1{}'$	$S_1{}' = (ab \cdot \bar{\beta} + (\bar{a} + \bar{b}) \cdot \beta) \cdot C_{in}$ $= (\bar{\alpha} \cdot \bar{\beta} + \alpha \cdot \beta) \cdot C_{in}$
18	$(S_2{}'' \to C_{in}) = C_{in}{}'$	$C_{in}{}' = (\bar{a} + \bar{b}) \cdot \beta + C_{in}$
19	$S_2 = 0$	$FALSE(S_2)$
20	$(C_{in}{}' \to 0) \equiv (C_{in}{}' \to S_2) = S_2{}'$	$S_2{}' = (ab + \bar{\beta}) \cdot \overline{C_{in}}$
21	$(\beta' \to S_2{}') = S_2{}''$	$S_2{}'' = ab \cdot \bar{\beta} + (ab + \bar{\beta}) \cdot \overline{C_{in}}$ $= \bar{\alpha} \cdot \bar{\beta} + (\bar{\alpha} + \bar{\beta}) \cdot \overline{C_{in}}$
22	$(\beta' \to C_{in}{}') = C_{in}{}''$	$C_{in}{}'' = ab \cdot \bar{\beta} + \left((\bar{a} + \bar{b}) \cdot \beta + C_{in}\right)$ $= \bar{\alpha} \cdot \bar{\beta} + (\alpha \cdot \beta + C_{in})$
23	$(C_{in}{}'' \to S_1{}') = S_1{}''$	$S_1{}'' = \left[(\alpha + \beta) \cdot \left((\bar{\alpha} + \bar{\beta}) \cdot \overline{C_{in}}\right)\right] + (\overline{\alpha \oplus \beta} \cdot C_{in})$ $= (\alpha \oplus \beta \cdot \overline{C_{in}}) + (\overline{\alpha \oplus \beta} \cdot C_{in})$ $= \alpha \oplus \beta \oplus C_{in} = \boldsymbol{Sum}$
24	$S_3 = 0$	$FALSE(S_3)$
25	$(S_2{}'' \to 0) \equiv (S_2{}'' \to S_3) = S_3{}'$	$S_3{}' = (\alpha + \beta) \cdot (\alpha \cdot \beta + C_{in})$ $= \alpha \cdot \beta + C_{in}(\alpha + \beta) = \boldsymbol{C_{out}}$

As mentioned earlier, the NAND gate output is the input(s) of the full adder cell in PPU_5 and PPU_6. Therefore, it is impossible to overlap the computational steps of the NAND gate and the full adder cell in the proposed serial n-bit signed array multiplier. The output equations of PPU_5 and PPU_6 are written in (17)–(20), respectively. The IMPLY-based PPU_5 and PPU_6 serial implementation algorithms are tabulated in Tables 16 and 17, respectively.

$$\mathbf{PPU_5}:\ Sum_{FA} = \overline{ab} \oplus \overline{cd} \oplus C_{in} \equiv \left[\left(\overline{(a \to \bar{b})} \to (c \to \bar{d})\right) \to \left(\left((a \to \bar{b}) \to \overline{(c \to \bar{d})}\right) \to C_{in}\right)\right]$$
$$\to \left[\overline{\left(\left(\overline{(a \to \bar{b})} \to (c \to \bar{d})\right) \to \overline{\left((a \to \bar{b}) \to \overline{(c \to \bar{d})}\right)}\right) \to \overline{C_{in}}}\right] \quad (17)$$

$$Cout_{FA} = \overline{ab} \cdot \overline{cd} + C_{in}(\overline{ab} + \overline{cd})$$
$$\equiv \left[\overline{\left(\overline{(a \to \bar{b})} \to (c \to \bar{d})\right) \to \left(\left((a \to \bar{b}) \to \overline{(c \to \bar{d})}\right) \to C_{in}\right)}\right] \quad (18)$$

$$\mathbf{PPU_6}:\ Sum_{FA} = \overline{ab} \oplus \beta \oplus C_{in} \equiv \left[\left(\overline{(a \to \bar{b})} \to \beta\right) \to \left(\left((a \to \bar{b}) \to \bar{\beta}\right) \to C_{in}\right)\right]$$
$$\to \left[\overline{\left(\left(\overline{(a \to \bar{b})} \to \beta\right) \to \overline{\left((a \to \bar{b}) \to \bar{\beta}\right)}\right) \to \overline{C_{in}}}\right] \quad (19)$$

$$Cout_{FA} = \overline{ab} \cdot \beta + C_{in}(\overline{ab} + \beta) \equiv \left[\overline{\left(\left(\overline{(a \to \bar{b})} \to \beta\right) \to \left(\left(\left((a \to \bar{b}) \to \bar{\beta}\right) \to C_{in}\right) \to 0\right)\right)}\right] \quad (20)$$

Using fixed inputs (constant bit '1') based on the Baugh-Wooley algorithm has made it possible to modify the output logic functions of the half adder (PPU_3) and the full adder (PPU_8). As a result, the number of computational steps of these two PPUs based on the IMPLY logic in the serial architecture is reduced to its minimum. If one of the inputs in (2) and (3) is considered equal to '1', the output functions are changed to (21) and (22).

$$Sum_{HA} = \alpha \oplus \beta = 1 \oplus \beta = \bar{\beta} \quad (21)$$
$$Cout_{HA} = \beta \cdot \alpha = \beta \quad (22)$$

In the classic structure, the implementation algorithm of the half adder cell takes twelve computational steps, but in the proposed structure of PPU_3, the number of computational steps is reduced to two steps (83% improvement). The implementation algorithm of the proposed PPU_3 is given in Table 18. According to the output equations of the full adder (see (6) and (7)), if one of the inputs of this cell is a constant bit '1', then the computational complexity of the output equations is reduced in this cell. By applying the constant bit '1' to (6) and (7), these output equations convert to (23) and (24), which are PPU_8's outputs. These outputs can be implemented serially based on the IMPLY logic in nine computational steps (59% improvement) with four memristors (two input memristors and two work memristors), as presented in Table 19.

$$Sum_{FA} = x \oplus y \oplus C_{in} = 1 \oplus y \oplus C_{in} = \overline{y \oplus C_{in}} \quad (23)$$
$$Cout_{FA} = x \cdot y + C_{in}(x + y) = y + C_{in} \quad (24)$$

The proposed n-bit signed CSA-based array multiplier's computational steps have been reduced from $(27n^2 - 24n + 24)$ to $(25n^2 - 32n + 1)$ based on the proposed implementation algorithms of PPU_1–PPU_8. The equation used to calculate the computational steps of the proposed signed array multiplier is written in (25). $CS_{SAmultiplier}$ refers to the number of computational steps of the signed array multiplier in (25). Furthermore, $5n - 4$ memristors are required to implement the n-bit signed CSA-based array multiplier, the same as its classic design.

$$CS_{SAmultiplier} = \#PPU_1 \times CS_{PPU1} + \#PPU_2 \times CS_{PPU2} + \#PPU_3 \times CS_{PPU3} + \#PPU_4 \times CS_{PPU4}$$
$$+ \#PPU_5 \times CS_{PPU5} + \#PPU_6 \times CS_{PPU6} + \#PPU_7 \times CS_{PPU7} + \#PPU_8 \times CS_{PPU8}$$
$$+ \#FA \times CS_{FA} + \#AND \times CS_{AND} \quad (25)$$

Table 18: The implementation algorithm of the proposed PPU_3 in the signed array multiplier.

Step	Operation	Equivalent Logic
1	$S_1 = 0$	$FALSE(S_1)$
2	$(\beta \to 0) \equiv (\beta \to S_1) = S_1{}'$	$S_1{}' = \bar{\beta} = \boldsymbol{Sum}$